What is GIT?

- Ameya Hattarke



Once upon a time long before git.? Work was done manually. In 1990's

Manual ownership Model

Each developer owned specific files or modules. No one else was allowed to touch each others code while work done

Shared Directory

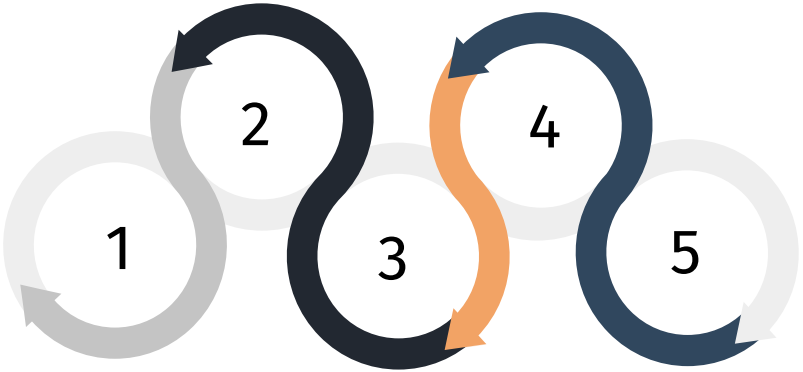
One main frame server File shared using terminal no branching no safety net

File Locking

While editing file user will lock the file other could just read it

Merging

Print code on paper to compare and manually combine changes



Code sharing

Email code magnetic tape or floppy disk will be sent to integrator

Issue in this system

Smaller codebases

Not suitable for huge code bases teams were smaller

Security risk

Sharing code and missing links

Time constraint

Release would take longer time as code merging was lengthy

Heavy Documentation

Who is working on what and what will go where documentation was complicated

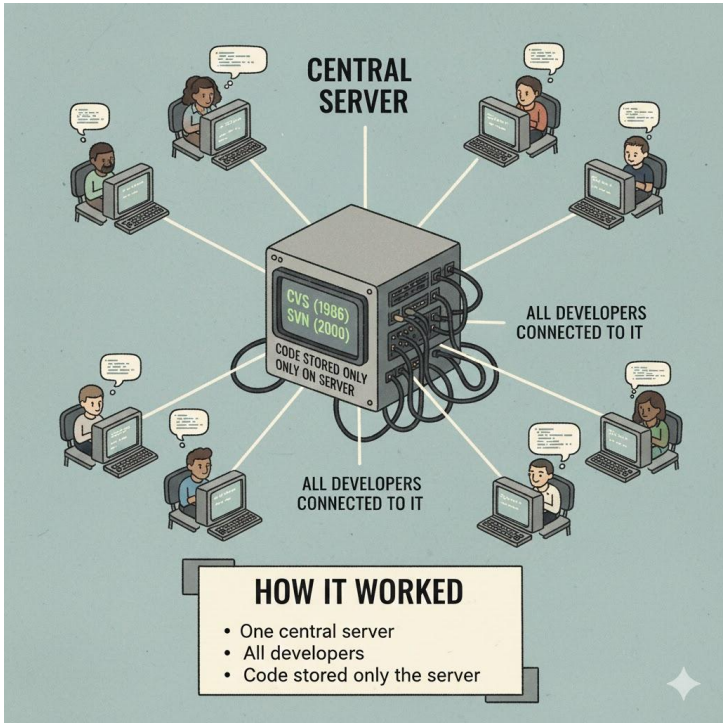
No rollbacks

No rollbacks or downshifting the version if file lost Gone forever

No collaboration

Parallel work was not supported.

Era of version control systems



01

1986: Concurrent version system
No atomic commit limited to file specific. Branching and versioning was weak.

02

Subversion control system 2000
Centralized architecture supports atomic commit still constant server access was needed server down no work can be done.

03

BitKeeper Early 2000's
First Distributed version control system Solve many problems for large projects like linux but it was proprietary closed source and license large products. Community conflicts as linux was major customer. Bit keeper revolutionized dvs but due to licensing developers stopped depending on it

Birth of the Git.

Scale problem

Linux kernel grew rapidly. Thousands of contributors were involved cvn svn failed

Bitkeeper was adopted 2002

It was fast and efficient for linux solved collaboration issue

Licensing conflict 2005

Bit keeper was proprietary. License restriction introduced community trust was broken expensive



No suitable alternatives

Existing tools were slow
Centralized systems were risky linux development couldn't stop

Birth of the git 2005

Created by linus torvalds in 10 days of coding(1st version).
Focused on speed and safety it is open source

Git became standard

Linux fully moved to git till 2006
Git hub was launched in 2008
Git became industry standard

What is git?



1. **Version Control System**
   Tracks every change in code. Allows rollback to any previous state
2. **Branching & Merging**
   Create separate branches for features Merges changes safely
3. **Collaboration**
   Multiple developers work in parallel. Prevents overwriting others work.
4. **Distributed systems**
   Full repository available locally. Works offline without server dependency.
5. **Speed & Rliability**
   Extremely fast operations ensure data integrity with hashing

Git

Version control tool

Installed on your local machine
Tracks file changes & history
Works **offline**
Open-source
Created by **Linus Torvalds (2005)**
Used for
Creating repositories
Committing code
Branching & merging

Git Hub

Cloud-based hosting platform

Built on top of Git
Stores repositories online
Enables **team collaboration**
Provides UI, PRs, issues, actions
Launched in **2008**
Used to create
Remote repositories

Pull Requests & code reviews

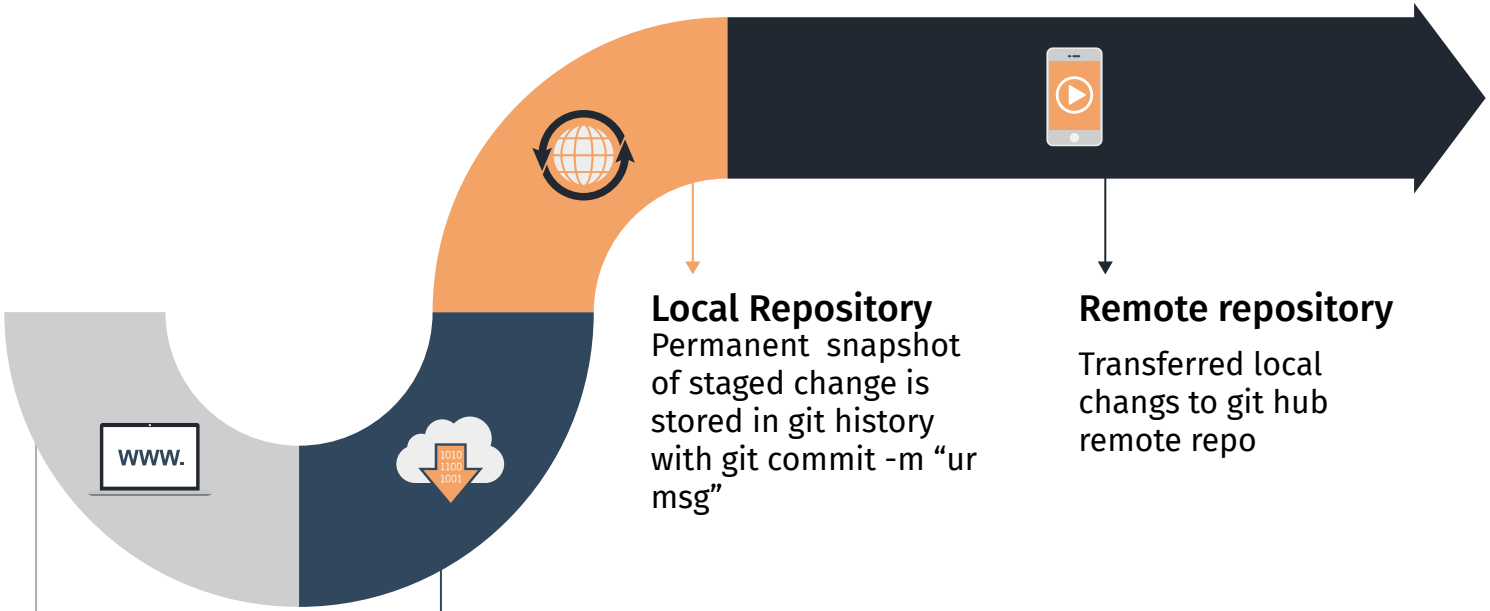
Project management

Before learning git vs After learning git.





How Git works



Your computer
Codes are written or your project on your system

Staging area
Where selected changes are prepared and controlled with Git add

Local Repository
Permanent snapshot of staged change is stored in git history with git commit -m “ur msg”

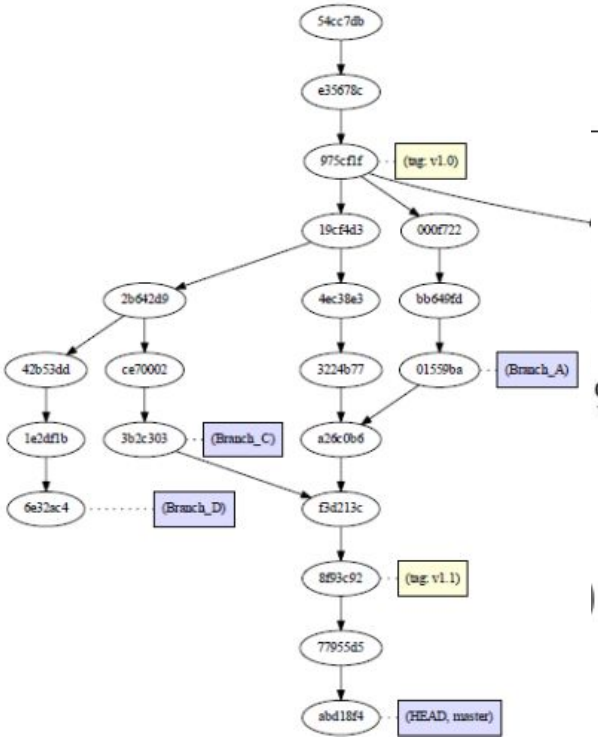
Remote repository
Transferred local changs to git hub remote repo

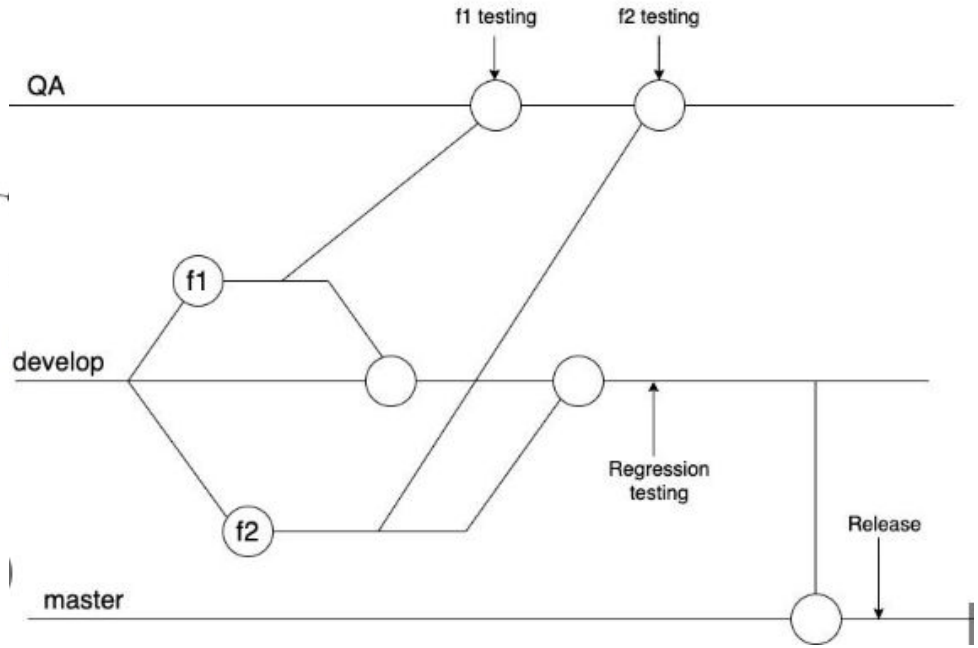
My first local commit

- You need Git Bash or Git installed on your computer.
- Open Git Bash / Terminal.
- Create or go to your project folder.
- Initialize Git in the folder using
    - **git init.**
- Git creates a local repository in the project.
- Create or modify files in the folder.
- Check file status using **git status**.
- Add files to the staging area using **git add .**.
- Commit the staged files using **git commit -m "First commit"**.
- Your code is now saved in the local Git repository.

Pushing in remote repo

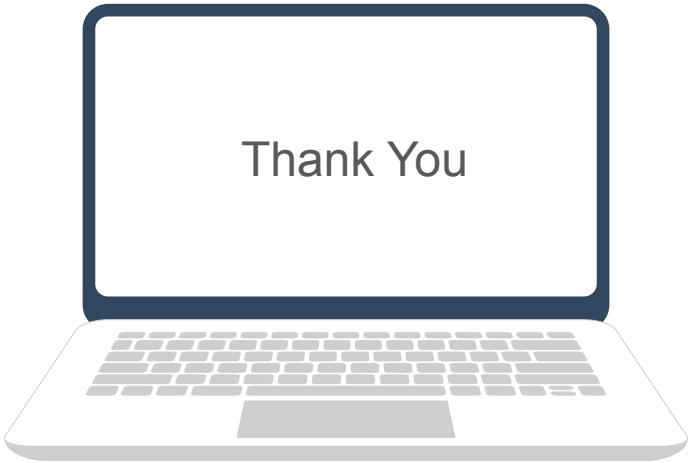
- Create a new repository on GitHub (do not add README).
- Copy the repository URL from GitHub.
- Open Git Bash / Terminal in your project folder.
- Ensure your code is already committed locally.
- Add GitHub as a remote using
- **git remote add origin <repo-url>.**
- Verify remote connection using git remote -v.
- Set the default branch using
- **git branch -M main.**
- Push code to GitHub using git push -u origin main.
- GitHub may ask for authentication (browser login/token).
- Your code is now available in the GitHub remote repository.


54cc7db
e35678c
975cf1f
(tag: v1.0)
19cf4d3
000f722
2b642d9
4ec38e3
bb649fd
42b53dd
ce70002
3224b77
01559ba
(Branch_A)
1e2df1b
3b2c303
(Branch_C)
a26c0b6
6e32ac4
(Branch_D)
f3d213c
8f93c92
(tag: v1.1)
77955d5
abd18f4
(HEAD, master)


f1 testing
f2 testing
QA
f1
develop
f2
Regression
testing
Release
master

Git overall

Branching Strategies (Git Flow, Trunk-Based)
Rebase vs Merge
Reset, Revert & Checkout (Deep Dive)
Git Stash
Cherry-Pick Commits
Resolving Merge Conflicts
Interactive Rebase
Git Log, Blame & Bisect
Tags & Release Management
Git Hooks
Detached HEAD State
Squash Commits
Git Internals (SHA, Objects)
Branch Protection Rules
Pull Request Best Practices


Thank You